Ders 10 Genişletilmiş Notu: Dizilerin Fonksiyonlara Aktarılması, Pointer'lar, Pointer Aritmetiği ve Frekans Hesabı

Ders 10 Genişletilmiş Notu: Dizilerin Fonksiyonlara Aktarılması, Pointer'lar, Pointer Aritmetiği ve Frekans Hesabı

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Amacı ve Algoritmalarda Kısıtlar
- 2. Dizilerin Fonksiyonlara Aktarılması
- 3. Kitaptaki Anket Örneği: Ortalama, Medyan ve Mod
- 4. İkili Aramayı `return` Kullanmadan Yazmak
- 5. Matrislerin Fonksiyonlara Aktarılması
- 6. Değerle Çağırmanın İki Sorunu
- 7. Çözüm: Adres Üzerinden İşlem Yapmak
- 8. Pointer Tanımı, İlk Değer ve Pointer Operatörleri
- 9. Pointer'larla Referansla Çağırma
- 10. Derste Yazılan Program: Swap, Dizi Yazdırma ve Prototipler
- 11. Pointer Aritmetiği
- 12. Pointer Aritmetiğinin Slayttaki Özeti
- 13. Bir Dizinin Sıralı Olup Olmadığını Bulmak
- 14. Frekans (Histogram) Hesabı

1. Dersin Amacı ve Algoritmalarda Kısıtlar

Hoca derse, bugün iki programlama konusu ve birkaç algoritma sorusu üzerinde çalışılacağını söyleyerek başladı. Bundan sonra algoritmaların akış diyagramını çizmekten çok kodunu yazmak istiyor. Kod yazmak için de programlama tarafındaki bazı eksiklerin giderilmesi gerekiyor, yoksa kod yazarken "çok fazla hareket kabiliyetimiz olmuyor". Dersin planı şudur: Daha önce laboratuvar bölümleriyle birlikte konuşulan diziler, geçen haftaki fonksiyonlarla ilişkilendirilecek, ardından pointer konusu işlenecek. Dersin sonunda da iki algoritma sorusu çözülecek.

Hoca algoritma geliştirme konusunda genel bir uyarı da yaptı. Bu tür derslerde çözümün önüne kısıtlar konur: "Bunu dizi kullanmadan yapın" ya da "bir tane diziyle yapın" gibi. Derste çözülen örneklerin amacı, başka problemlere de fikir vermektir. Bir kişinin önceki bilgileriyle geliştirdiği algoritma bu sınırları ihlal ediyorsa, yani kurallara uymuyor ya da verimsizse, algoritma istenen şeyi karşılamamış olur. Daha iyi bir çözüm üretilirse bunda sorun yoktur, ama beklenen seviyenin altında kalan bir çözüm kabul edilemez.

> **Hocanın vurgusu:** "Ben böyle biliyordum" bir gerekçe değildir. Algoritma sorularında önemli olan, soruda söylenen kısıtlardır.

Hoca ayrıca ilerideki bir konuya işaret etti: Gelecek haftadan sonra algoritmaların analizine girilecek. Derste arada sorulan "kaç adım gerçekleşiyor" sorusu ve "n adım, n^2 adım" gibi ifadeler daha sistemli bir hâle getirilecek.

2. Dizilerin Fonksiyonlara Aktarılması

Hoca dizileri baştan anlatmadı, yalnızca hatırlattı: Dizi bellekte sıralı olarak saklanır ve derleyici sıfırıncı elemanın adresini bilir. Diziler sayesinde, özellikle en büyük sayıyı bulma probleminden yola çıkan pek çok problem daha kolay çözülür, çünkü veri program kapanana kadar bellekte saklanabilir. Elemanlara erişim, tanımlama, ilk değer verme ve karakter dizileri daha önce konuşulduğu için tekrar edilmedi. Konuşulmamış olan kısım, dizilerin fonksiyonlara aktarılmasıdır.

2.1 Bütün Dizi ve Tek Eleman

Diziler fonksiyonlara call by reference mantığıyla aktarılır. Yani bir fonksiyona dizinin etiketi (adı) gönderildiğinde, dizi fonksiyon içinde değiştirilirse bu değişiklik fonksiyonun çağrıldığı yere yansır, orijinal dizi değişir. Çağıran yerin `main` olması gerekmez, başka bir fonksiyon da olabilir.

Dizinin tek bir elemanını göndermek ise farklıdır. Bu, normal bir değişkeni fonksiyona aktarmakla aynıdır, yani call by value'dur. Bir `x` ya da `y` değişkeni veya 3, 5 gibi bir sayı göndermekten farkı yoktur. Fonksiyonda yapılan değişiklik çağıran yere yansımaz.

2.2 Fonksiyonun Tanımı

Örneğin `modifyArray` adlı bir fonksiyon, çağrıda diziyi ve boyutunu alsın: `modifyArray(dizi, 10)`. Fonksiyon tanımında dizi parametresi köşeli parantezle ve parantez içi boş bırakılarak yazılır. Boyut, ayrı bir `int size` parametresidir:

```
void modifyArray(int dizi[], int size) {
    int i;
    for (i = 0; i < size; i++) {
        /* dizi[i] ile main'de yazar gibi islem yapilir */
    }
}
```

Dizinin bütün elemanlarına erişmek için `i`, 0'dan `size`'a kadar gider. Elemanlar üzerindeki işlemler `main`'de nasıl yazılıyorsa burada da öyle yazılır.

2.3 Dizinin Adı ve Adresi

Slayttaki bir örnekte dizinin adı üç farklı biçimde yazdırılıyor: `array`, `&array[0]` ve `&array`. Dizi etiketi yazıldığında bir adres yazdırılır. Sıfırıncı elemanın adresi de adres operatörüyle yazdırılır. Dizinin adresi dersek de aynı sonucu alırız. Üçü de aynı adresi yazdırır.

> **Hocanın vurgusu:** `array`, `&array[0]` ve `&array` aynı adresi yazdırır. Bu önemli bir bilgidir.

2.4 `modifyArray` ve `modifyElement`

Kitap örneğinde önce orijinal dizi yazdırılıyor, sonra `modifyArray` çağrılıyor, sonra dizi tekrar yazdırılıyor. `modifyArray` diziyi 0, 2, 4, 6, 8 olacak biçimde dolduruyor ve bu değişiklik `main`'de görülüyor. Tek elemanla çalışan `modifyElement` ise elemanı 2 ile çarpıyor, ama bu çarpım fonksiyonun içinde kalıyor. Hocanın daha önce birlikte yapıldığını hatırlattığı "6, 12, 6" sonucu şunu gösterir: Eleman önce 6'dır, fonksiyon içinde 12 olur, `main`'e dönünce yine 6'dır. Sebep, tek elemanın call by value ile gitmesidir.

Parametrenin önüne `const` gelmesi gibi durumlar bu derste konuşulmadı.

3. Kitaptaki Anket Örneği: Ortalama, Medyan ve Mod

Hoca, sıralama fonksiyonunun fonksiyon olarak yazılma sebebini hatırlattı: Bubble sort kodu sürekli çağrılmak istenebilir, tek bir yere yazmak yerine fonksiyon olarak çağırırız. Ardından kitaptaki örnek çalışmaya (case study) geçti ve bu örneği çalışıp kodunu yazmamızı istedi.

Örnekte bir dizinin ortalaması, medyanı ve modu bulunuyor. Her hesap için hoca önce "kaç işlem gerekir?" sorusunu sordu.

3.1 Ortalama

n elemanlı bir dizinin ortalaması için bütün elemanlar toplanır ve eleman sayısına bölünür.

Soru: Ortalama için kaç işlem gerekir? **Cevap:** n işlem. Önce bütün elemanlar toplanır, sonra toplam n'e bölünür.

Soru: Ortalama hesabı için dizinin sıralı olması gerekir mi? **Cevap:** Hayır. Doğrudan toplanır, iş tek döngüyle biter.

3.2 Medyan

Medyan, orta sıradaki elemandır ve dizinin sıralı olmasını gerektirir. Bu yüzden önce sıralama yapılır, yani önce bir sıralama maliyeti vardır. Şu ana kadar görülen bubble sort ve selection sort için bu maliyet n^2'dir. Sonra n elemanlı `D` dizisinin $n/2$ indisli elemanı alınır.

Hocanın örneği 5 elemanlı bir diziydi:

- Sıralama, en kötü ihtimalle $5^2 = 25$ işlem yapar.
- Orta indis $5/2$'dir. C'de tam sayı bölmesi alta yuvarlar, sonuç 2'dir.
- 2 indisli eleman ortadaki elemandır. Hocanın örneğinde bu değer 3'tür.

  Ek bilgi (derste anlatılmadı): Derste `size / 2` formülü tek sayıda eleman için gösterildi. Eleman sayısı çift olduğunda ortada tek bir eleman yoktur. Bu durumda medyan olarak büyük olasılıkla ortadaki iki elemanın ortalaması alınır.

Kitaptaki kodda medyan fonksiyonu gelen `answer` dizisini doğrudan sıralama fonksiyonuna gönderiyor. Sıralamadan sonra dizinin içeriğinin değişmesinin sebebi call by reference mekanizmasıdır. Yeni bir dizi oluşturulmaz. Bellekte tek bir dizi vardır, elemanlar sıralama algoritmasının mantığına göre kendi aralarında yer değiştirir ve orijinal dizi değişmiş olur.

3.3 Mod ve Frekans Dizisi

Mod, dizide en sık geçen elemandır. Bunu bulmak için önce her değerden kaç tane olduğu bulunmalıdır: Kaç tane 1 var, kaç tane 2 var, ..., kaç tane 9 var? Yani elemanların frekansı ya da histogramı çıkarılır. En yüksek sayıda olan değer dizinin modudur.

Bu iş için yeni bir veri yapısı gerekir. Elemanlar zaten bir dizide saklandığı için bir dizi daha oluşturulur ve her elemandan kaçar tane olduğu orada sayılır. Bu yüzden `mode` fonksiyonuna hem frekans dizisi hem orijinal dizi gönderilir. Mod hesabı için dizinin içeriğini, yani elemanların hangi aralıkta değiştiğini de bilmemiz gerekir.

Kitaptaki `mode` fonksiyonu önce frekans dizisini sıfırlıyor. Hoca bunun gerekli olmadığını, çünkü dizinin yukarıda zaten sıfırlandığını belirtti. Sonra her bir eleman tek seferde sayılıyor. Bu tek seferde sayma işinin dersin sonunda ayrıca konuşulacağını söyledi (Bölüm 14). Ardından frekans dizisinde en fazla geçen değer aranıp yazdırılıyor.

Kitapta işlemlerin her biri ayrı bir fonksiyondur: ortalama, medyan, mod, bubble sort, dizi yazdırma. Yazdırma, sıralamanın içine konmamıştır. Hoca bunun sebebinin geçen derste konuşulduğunu hatırlattı. Veri de klavyeden alınmıyor, bir `response` dizisine ilk değer olarak veriliyor. Bu dizi slaytta matris gibi görünse de tek boyutlu bir dizidir.

Aşağıdaki fonksiyonlar, hocanın anlattığı akıştan kuruldu ve ders sonrasında `gcc -Wall` ile derlenip denendi:

```c
#include <stdio.h>
#define SIZE 99

void bubbleSort(int a[]);

double mean(int answer[]) {
    int j, total = 0;
```

```
    for (j = 0; j < SIZE; j++)
        total += answer[j];
    return (double) total / SIZE;
}

int median(int answer[]) {
    bubbleSort(answer);        /* orijinal dizi siralanir */
    return answer[SIZE / 2];
}

int mode(int freq[], int answer[]) {
    int rating, j, largest = 0, modeValue = 0;
    for (rating = 1; rating <= 9; rating++)
        freq[rating] = 0;
    for (j = 0; j < SIZE; j++)
        ++freq[answer[j]];     /* tek seferde sayma */
    for (rating = 1; rating <= 9; rating++)
        if (freq[rating] > largest) {
            largest = freq[rating];
            modeValue = rating;
        }
    return modeValue;
}

void bubbleSort(int a[]) {
    int pass, j, hold;
    for (pass = 1; pass < SIZE; pass++)
        for (j = 0; j < SIZE - 1; j++)
            if (a[j] > a[j + 1]) {
                hold = a[j];
                a[j] = a[j + 1];
                a[j + 1] = hold;
            }
}
```

3.4 Görüntü İşlemede Histogram

Hoca, frekans hesabının görüntü işlemede de kullanıldığını söyledi. Bir görüntünün çok mu açık, çok mu karanlık, yoksa düzgün dağılımlı mı olduğu gibi bilgiler çıkarılırken histogram bir özellik olarak kullanılır. Piksel değerleri 0-255 arasında kodlansın, 0 siyah, 255 beyaz olsun. Ortalama ya da medyan 0'a yakınsa görüntü karanlıktır. 255'e yakınsa, yani 200, 225, 230 gibi değerlerdeyse görüntü aydınlık, çok fazla ışıklandırılmıştır. Moda ise ayrıca bakılır.

3.5 `SIZE` Sabiti ve `#define` Hakkında Bir Soru

Örnekte `SIZE`, 99 olarak tanımlıdır ve her fonksiyona bu sabit üzerinden iş yaptırılır. Bu yüzden ayrıca boyut hesabına gerek yoktur.

> **Ders sonrası not:** Hoca burada `SIZE`'ın `const` ile belirtildiğini söyledi, hemen sonra ise `define` ile tanımlanmış gibi anlattı. Anlatımın geri kalanına göre slaytta büyük olasılıkla `#define SIZE 99` yazıyor. `#define` bir yerine koymadır, `const` ise bir değişken niteleyicisidir.

Soru: `SIZE` yerine bir tam sayı değil de örneğin `Ahmet` yazsaydık ne olurdu, derleyici bunu nasıl algılar? **Cevap:** `#define` bir yerine koymadır (substitution). Program derlenmeden önce `SIZE` görülen her yere karşısındaki değer kopyalanır. Burada hiçbir tip kontrolü yapılmaz. `Ahmet` yazılsaydı, önce bu metin `SIZE` geçen yerlere kopyalanır, sonra o yerde hata alınırdı, çünkü `Ahmet`'ten bir boyut olmaz. Kontrol, kodu yazan kişiye aittir.

> **Ders sonrası not:** Hoca bu hatanın "programın çalışması esnasında" alınacağını da söyledi. Hata derleme sırasında alınır, program çalışmaya başlamaz. Ders sonra-

sında `#define SIZE Ahmet` ve `int a[SIZE]; gcc -Wall` ile denendi: `'Ahmet' undeclared` derleme hatası, `a` dizisinin tanımlandığı satırda verildi.

3.6 Bubble Sort Hatırlatması

Hoca kitaptaki bubble sort'u "fena bir bubble sort değil" diye tanıttı. Dış döngü 1'den başlayıp `size`'a kadar gider. Sıfırdan da başlayabilirdi, ama son kontrolü yapmaya gerek yoktur, çünkü en büyük eleman zaten sona gönderilmiş olur. İç döngü `size - 1`'e kadar gider.

Algoritmanın mantığını 4, 3, 8, 6 dizisiyle gösterdi. Birinci turda komşular sırayla karşılaştırılır:

- 4 ile 3 karşılaştırılır, yer değiştirir: 3, 4, 8, 6.
- 4 ile 8 karşılaştırılır, yer değiştirmez: 3, 4, 8, 6.
- 8 ile 6 karşılaştırılır, yer değiştirir: 3, 4, 6, 8.

Birinci turun sonunda en büyük eleman kesinlikle en sona ötelenmiş olur. Bir sonraki turda sona tekrar bakmaya gerek yoktur. Her tur sıfırdan başlamak zorundadır. Ama her zaman bütün kontrolleri yapmak gerekmez: Bu örnekte dizi birinci turda sıralanmıştır, kod ise 2. ve 3. turları da yapar. Bunu önlemek için ek bir bayrak (flag) değişkeni kullanılır ve algoritma biraz hızlandırılır. Hoca bu ayrıntıya sonra döneceğini söyledi.

> **Ders sonrası not:** Hoca, `size - 1` sınırının sondaki elemana tekrar bakmamayı sağladığını söyledi. Sabit `size - 1` sınırı ise yalnız `a[j + 1]` erişiminin dizinin dışına çıkmasını önler, her turda son elemana yine bakılır. Yukarıdaki kod 4, 3, 8, 6 için python ile izlendi: 3 turda toplam $3 \times 3 = 9$ karşılaştırma yapıldı, dizi ilk turda 3, 4, 6, 8 oldu. Yerine oturan elemanları gerçekten atlamak için iç döngünün sınırı her turda daralmalıdır (`j < size - pass`).

4. İkili Aramayı `return` Kullanmadan Yazmak

Slaytlarda önce doğrusal arama (linear search) fonksiyonu, sonra ikili arama (binary search) var. Doğrusal arama fonksiyonu bir tam sayı dizisi, aranan eleman ve boyut alıyor. İkili aramada `low` ve `high` sınırları tutuluyor, dizi ikiye bölünüyor. Her adımda dizinin bir yarısı elenir. İşlem sayısı en fazla $\log n$ mertebesindedir, hoca bazen bunun bir fazlası olabildiğini de söyledi. Kitaptaki çıktı, her adımda hangi elemana bakıldığını canlandırır gibi gösteriyor: sol eleman, sağ eleman ve `middle` noktası her adımda kayıyor.

Kitaptaki kodda eleman bulunduğunda döngü içinden `return` yapılıyor. Hoca bunu `return` olmadan yazmayı gösterdi. Çünkü bu aşamada kodun ayrıntılarını didiklemek istiyor. Fikir, "aranan değer `b[middle]`'dan farklı" şartını döngünün şartına taşımaktır. Ama bu şart `middle`'ı kullandığı için `middle`'ın şart kontrol edilmeden önce bir değeri olmalıdır. Hoca iki yol söyledi:

- `while`'dan önce `middle`'a bir değer vermek, örneğin sıfır, ya da `middle`'ı döngüden önce hesaplamak.
- `do-while` kullanıp şartı döngünün altına indirmek. Hoca kendi kodunu da böyle yazmış.

Hoca bu örneğin hem programlama hem algoritma tarafını güzel gösterdiğini söyledi. `do-while` hâli şöyle yazılabilir:

```
int binarySearch(const int b[], int key, int low, int high) {
    int middle;
    do {
        middle = (low + high) / 2;
        if (key < b[middle])
            high = middle - 1;
        else if (key > b[middle])
            low = middle + 1;
    } while (low <= high && key != b[middle]);

    if (key == b[middle])
        return middle;
    return -1;
}
```

> **Ders sonrası not:** Altyazıda yalnız `key != b[middle]` şartının taşınmasından söz ediliyor. Aranan değer dizide yoksa bu şart tek başına döngüyü bitirmez, bu yüzden `low <= high` şartı da kalmalıdır. Yukarıdaki fonksiyon `gcc -Wall` ile derlendi. 7 elemanlı sıralı bir dizide hem bulunan hem bulunmayan değerler denendi ve hepsinde doğru sonuç verdi.

5. Matrislerin Fonksiyonlara Aktarılması

Geçen derste görülmeyen bir konu, çok boyutlu dizilerin fonksiyonlara gönderilmesidir. Fonksiyon tanımında parametrenin iki boyutlu olduğu iki çift köşeli parantezle söylenir. Hocanın kuralı şudur: Kaç sütun olduğu yazılmak zorundadır, yoksa derleyici hata verir. Satır bilgisi yazılmak zorunda değildir.

```
void printArray(int a[][3]) { ... }
```

Slayttaki fonksiyonda satır döngüsü 0'dan 1'e, sütun döngüsü 0'dan 2'ye gidiyor. Çünkü dizinin 2 satır ve 3 sütunu olduğu ilk değer vermeden biliniyor. Burada 1 sabit yazılmış. Fonksiyonu daha parametrik hâle getirmek için satır sayısı ikinci bir parametre olarak gönderilir, 1 yerine bu parametreye kadar gidilir. Hoca, istenirse sütun sayısının da ayrı parametreyle alınabileceğini söyledi. Geri kalan her şey, matris elemanına normal erişimle aynıdır.

```
#include <stdio.h>

void printArray(int a[][3], int row) {
    int i, j;
    for (i = 0; i < row; i++) {
        for (j = 0; j < 3; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }
}
```

Ders sonrasında sütunun yazılmadığı `void f(int a[][]);` bildirimi `gcc -Wall` ile denendi ve `array type has incomplete element type` derleme hatası alındı. Hocanın söylediği doğrulanmış oldu.

> **Ek bilgi (derste anlatılmadı):** Sütun sayısının zorunlu olmasının sebebi büyük olasılıkla adres hesabıdır. Matris bellekte satır satır, art arda durur. Derleyici `a[i][j]` için başlangıçtan kaç eleman ileri gideceğini $i \times \text{sütun} + j$ ile bulur. Bu hesapta sütun sayısı gerekir, satır sayısı gerekmez.

5.1 Öğrenci Notları Örneği

Kitaptaki bir diğer örnek bu yapıyı daha parametrik kullanıyor. Öğrenci sayısı ve sınav sayısı makrolarla verilmiş: `#define STUDENTS 3` ve `#define EXAMS 4`. Yani 3'e 4'lük bir not matrisi var. Matris `minimum`, `maximum` ve yazdırma fonksiyonlarına gönderiliyor. Parametrede sütun sayısı (`EXAMS`) verilmek zorundadır, satır sayısı verilmek zorunda değildir. Bunun yerine öğrenci sayısı `pupils`, sınav sayısı `tests` adlı değişkenlerle ayrıca gönderiliyor.

Fonksiyon içinde iki iç içe döngüyle matris geziliyor. Hoca kopyalama mekanizmasını sordu.

Soru: `STUDENTS` ve `EXAMS` değerleri `pupils` ve `tests`'e hangi mekanizmayla kopyalanır? **Cevap:** Call by value. Bunlar düz değişkenlerdir. Üstelik `STUDENTS` ve `EXAMS` `define` ile tanımlı olduğu için değiştirilmek istense de değiştirilemez. Derleme öncesinde bunların yerine sabit olarak 3 ve 4 gelir, çağrıda da bu değerler parametrelere kopyalanır.

```
#define STUDENTS 3
#define EXAMS 4

int minimum(int grades[][EXAMS], int pupils, int tests) {
    int i, j, lowGrade = grades[0][0];
    for (i = 0; i < pupils; i++)
        for (j = 0; j < tests; j++)
```

```
            if (grades[i][j] < lowGrade)
                lowGrade = grades[i][j];
    return lowGrade;
}
```

> **Ders sonrası not:** Altyazıda hoca döngüyü anlatırken "pupil sayısına kadar gidiyor, 3 göndermiştik, pardon 4 göndermiştik" diyor. Tanımlara göre doğrusu, `pupils` 3 ve `tests` 4'tür. Hemen sonra hoca da değerleri "students ve exam, yani 3 ve 4" diye söylüyor.

Hoca bu kısmı, geçen dersten farklı olarak matrislerin fonksiyonlara nasıl aktarıldığının görüldüğünü ve sütun bilgisinin söylenmesi gerektiğini tekrar ederek bitirdi.

6. Değerle Çağırmanın İki Sorunu

Programı modüler hâle getirmek için fonksiyonlara bölüyoruz, dizileri gönderince içerikleri fonksiyonda değişebiliyor. Ama bugünkü bilgilerle fonksiyon kullanımında çözülemeyen iki temel problem var. Hoca bunu öğrencilere sordu ve önce problemin ne olduğunun bilinmesi gerektiğini söyledi.

Call by reference dizilerde olur, normal değişkenlerde ise varsayılan olarak call by value vardır. Hoca şu farkı tahtada gösterdi: `int a` bir değişken, `int b[...]` bir dizi olsun. `f` fonksiyonuna `a`, `g` fonksiyonuna `b` gönderilsin. `f`'in tanımında parametre `int a`, `g`'nin tanımında ise dizi parametresi olur. Dönüş tipi burada önemli değildir. Soru şudur: `a` gibi bir değişkeni, bir karakter ya da `float` değişkeni fonksiyona gönderdiğimizde içeriğinin değişmesini nasıl sağlarız?

Hoca geçen derslerdeki swap örneğine döndü:

```
void swap(int a, int b) {
    int t;
    t = a;
    a = b;
    b = t;
}
```

`main` içinde `int x = 5, y = 8;` olsun ve `swap(x, y)` çağrılsın. Çağrıdan önce değerler 5 ve 8'dir.

Soru: Çağrıdan sonra `x` ve `y`'nin değerleri ne olur? **Cevap:** Değişmez, çünkü değerle çağırıyoruz ve fonksiyon `void` olduğu için bir değer de döndürmüyor.

Soru: `return` yapsaydık ne olurdu? **Cevap:** Fonksiyonu `int` döndürecek biçimde yazsak bile en fazla tek bir değer, örneğin değişmiş `a` döndürülebilir. Çağıran yerde bu değer bir değişkene atanabilir, ama öbür değer alınamaz. Normalde swap `void` yazılır.

Böylece iki problem ortaya çıkmış oldu. Hoca bunları İngilizce olarak yazdı:

- **Birinci problem:** Orijinal değerler (original values) herhangi bir fonksiyonun içinde değiştirilemez.
- **İkinci problem:** Geri döndürülebilecek değer sayısı en fazla birdir ("number of return is one"). İki, üç değer döndürülemez.

Çözüm call by reference'tır. Ama C'deki call by reference gerçek bir call by reference değildir, çözüm pointer kullanmaktır. Bir fonksiyona bir, iki, üç ya da yirmi değer gönderip içeriklerinin değişmesini istiyorsak call by reference mekanizmasını dolaylı biçimde kullanırız. Hoca, pointer'ın bir algoritma konusu değil, doğrudan bir programlama konusu olduğunu söyledi.

7. Çözüm: Adres Üzerinden İşlem Yapmak

Slaytta pointer'ların call by reference mekanizmasını "simüle ettiği" yazıyor. Simüle etmek, farklı ve dolaylı bir şekilde, değişkenlerin bellekteki adresleri üzerinden işlem yapmak demektir. Pointer'ların dizilerle, string'lerle ve fonksiyonlarla çok ilişkisi vardır.

Aslında biz pointer mantığını zaten kullanıyoruz:

- `scanf("%d", &a)` call by reference'tır. `scanf`'e `a`'nın adresi gider.

- `printf("%d", a)` call by value'dur. `printf` ekrana yazar, bir şeyin değerini değiştirmesi beklenmez.

Hoca bunu bellekte şöyle anlattı. `main` içinde değeri 5 olan bir `a` değişkeni olsun. `printf` ya da herhangi bir `f` fonksiyonu `a` ile çağrıldığında, fonksiyonun kendine ait bir `a`'sı olur ve 5 oraya kopyalanır. Fonksiyon içeride isterse 1000 artırsın, isterse 500 çıkarsın, `main`'e dönüldüğünde `a` olduğu gibi kalır. `scanf`'te ise `a`'nın adresi, diyelim ki T, fonksiyona gönderilir. Fonksiyon önce bu adrese bakar, T numaralı adrese gider ve oradaki değeri değiştirir. Yani dolaylı bir değişim vardır.

Pointer konusunun özü şudur: Bir değişkenin adresini elde edip değişkeni bu adres üzerinden manipüle etmek. Bunu aynı işi iki yoldan yaparak gösterdi. `int x = 5;` için `x++` işlemi değişkeni kendi etiketi üzerinden değiştirir. Aynı iş adres üzerinden de yapılabilir:

```
int x = 5;
int *px;
x++;          /* etiket uzerinden */
px = &x;      /* x'in adresi px'e kopyalanir */
(*px)++;      /* adres uzerinden: ayni is */
```

`px`'in tipi `int` pointer'dır. `int` ile `int *` farklı tiplerdir, bu yüzden `px = x;` ya da `x = px;` yazılamaz. Tipleri uyumlu hâle getirmek için adres operatörü `&` kullanılır: `&x`, `int *` tipinde bir değer döndürür. Sol taraf da sağ taraf da `int *` olduğu için `px = &x;` ataması sağlıklıdır. Ders sonrasında `px = x;` denendi, `gcc -Wall` bu derleyici sürümünde (gcc 15) tipi uyuşmayan atamayı derleme hatası olarak verdi.

`(*px)++` şu anlama gelir: `px`'in gösterdiği yerdeki değeri al, ona bir ekle, eklediğin değeri tekrar `px`'in gösterdiği yere yaz. Bir şeyin adresini alıp bir değişkene atamak **referans alma (referencing)** işlemidir. Referanstan tekrar değere gitmek ise bu işlemin tersi, yani **referanstan çıkarma (dereferencing)** işlemidir. `px` yerine `&x` konup `*&x` yazılırsa `*` ile `&` birbirini götürür, geriye `x` kalır. Yani `x` üzerinde doğrudan işlem yapmaktan farkı kalmaz. Ama adres üzerinden erişebilmek, fonksiyonlarda ve başka yerlerde pek çok problemi çok daha kolay çözmemizi sağlar.

8. Pointer Tanımı, İlk Değer ve Pointer Operatörleri

8.1 Tanımlama

Normalde değişken adına özel karakter olarak yalnız alt çizgi eklenebilir. Pointer tanımındaki `*` değişken adının parçası değildir. Değişken adı `px`'tir, önündeki yıldız bunun pointer tipinde olacağını, `int` de pointer'ın türünü söyler. Bir `char` değişkenine pointer tanımlanacaksa pointer'ı da `char *` olur. Bu her tip için yapılabilir.

```
int *px;
char c;
char *pc;
```

Hoca isimlendirmenin zorunlu olmadığını, ama `pc` gibi anlamlı adlar seçmenin iyi olacağını söyledi.

8.2 İlk Değer

Pointer'lara ilk değer olarak 0, büyük harflerle `NULL` ya da herhangi bir adres verilebilir. Örneğin `char c;` tanımından sonra `char *pc = &c;` doğrudan bir adres atamasıdır.

8.3 Adres Operatörü: `y` ve `yPtr` Örneği

> **Hocanın vurgusu:** Bu slayt, konunun en önemli slaytlarından biridir. Bu basamakları sağlam çıkan, pointer konusunda çok sıkıntı yaşamaz.

Adım adım:

1. `int y = 5;` Bellekte `y` için bir yer ayrılır, içinde 5 vardır.

2. `int *yPtr;` `yPtr` için de bir yer ayrılır. O da bir değişkendir, bellekte yer tutmak zorundadır ve kendine ait bir adresi vardır. Henüz içinde y'nin adresi yoktur.
3. `yPtr = &y;` y'nin adresi `yPtr`'ye kopyalanır. Aradaki bağ böyle kurulur: `yPtr`, y'nin bulunduğu bellek hücresini gösterir. y'nin varsayımsal adresi 600000 ise `yPtr`'nin içinde artık 600000 vardır.
4. Değeri 10 yapmak için iki yol vardır: `y = 10;` ya da `*yPtr = 10;`.

`main` içinde, yani değişkenin kendi kaynağında tercih ettiğimiz yol `y = 10;` yazmaktır. Ama işin içine başka bir fonksiyon girerse `*yPtr = 10;` biçimini kullanmak zorundayız.

8.4 İçerik Operatörü ve Birbirini Götürme

Adres operatörünün tersi, içerik operatörüdür (indirection ya da dereferencing). Slayttaki ifadeler şöyledir: `&` operandının adresini döndürür. `*` ise operandının gösterdiği şeyin eş anlamlısını (synonym, alias) döndürür. Yani döndürdüğü şey, o adresin gösterdiği yerdeki değerdir. Daha önce `yPtr` y'yi işaret ettiği için `*yPtr` bize y'yi verir.

`*` ve `&` birbirinin tersidir ve birbirini iptal eder. `int y = 10;` için:

- `&y`, y'nin adresidir.
- `*&y`'de ikisi birbirini götürür, `y` kalır.
- `&*&y`'de iç taraftaki ikisi birbirini götürür, tekrar y'nin adresi kalır.
- `*&*&y`'de yine birbirlerini götürürler, tekrar `y` kalır.

Hoca bunun daha ayrıntılı kısmının gelecek dönem yapısal programlamada görüleceğini söyledi.

8.5 Kitap Örneği: `a` ve `aPtr`

`int a;` ve `int *aPtr;` tanımlanıyor. Bunlar bildirimdir, henüz atama yoktur. Sonra `a = 7;` ve `aPtr = &a;` yapılıyor.

```c
#include <stdio.h>

int main() {
    int a;
    int *aPtr;

    a = 7;
    aPtr = &a;

    printf("%p %p\n", (void *)&a, (void *)aPtr);
    printf("%d %d\n", a, *aPtr);
    printf("%p %p\n", (void *)&*aPtr, (void *)*&aPtr);
    return 0;
}
```

- `&a` ve `aPtr` aynı adresi yazdırır: işletim sistemi o anda a'ya neresini ayırdıysa o adres.
- `a` doğrudan 7 yazdırır. `*aPtr` de `aPtr`'nin gösterdiği yere, yani a'nın adresine gider ve oradaki 7'yi yazdırır.
- `&*aPtr` ve `*&aPtr`'de operatörler birbirini iptal eder, ikisi de `aPtr`'dir. `aPtr` bir bellek adresi olduğu için ilk satırdaki sonucun aynısı görülür.

Slayttaki çıktıda adres 0012FF7C'dir. Hoca bunun rastgele bir değer olduğunu, kod kendi bilgisayarımızda çalıştırıldığında farklı adresler görülmesinin normal olduğunu söyledi. Ders sonrasında kod `gcc -Wall` ile derlenip çalıştırıldı: Üç satırda da adresler kendi aralarında aynı, değerler 7 ve 7 çıktı.

Slayttaki operatör tablosunda parantez, işlem önceliği ve dizi işlemleri gibi diğer operatörler de var. Hoca bunları şimdilik geçti.

9. Pointer'larla Referansla Çağırma

9.1 Sayıyı İki ile Çarpma

Bir fonksiyona bir şey gönderip değişikliğin çağıran yere yansımasını istiyorsak fonksiyona gelecek şey değişkenin adresi olacaktır. Adres geleceği için parametre pointer tipinde tanımlanır:

```c
#include <stdio.h>

void doubleNumber(int *number) {
    *number = 2 * (*number);
}

int main() {
    int z = 10;
    doubleNumber(&z);
    printf("%d\n", z);    /* 20 */
    return 0;
}
```

`number`'ın gösterdiği yerdeki değer 2 ile çarpılır ve yine oraya atılır. Çağrıda `z` değil `&z` yazılır, çünkü parametre `int *`'dır ve `z` ile çağırmak hata verir. `doubleNumber(&z)` aslında `number = &z` demektir: `z`'nin adresi `number`'a gelir. Sonuçta `z`'nin değeri `main` içinde 10'dan 20'ye değişir.

Sayısal olarak: $z = 10$, $*number = 2 \times 10 = 20$, `main`'de $z = 20$.

> **Ders sonrası not:** Hoca fonksiyonun adını "double" diye söyledi. `double` C'de bir anahtar sözcüktür ve fonksiyon adı olamaz. `void double(int *number)` ders sonrasında `gcc -Wall` ile denendi ve derleme hatası verdi. Bu yüzden yukarıda `doubleNumber` yazıldı. Bu kod aynı derleyiciyle derlendi ve 20 yazdırdı. `int *` parametreye `z`'yi göndermek ise gcc 15'te derleme hatası verdi, `-std=c89` ile yalnız uyarı verdi.

9.2 `cubeByValue` ve `cubeByReference`

Hoca, buraya kadar konuşulan şeylerin birinci problemi, yani orijinal değerin değiştirilmesini ifade ettiğini söyledi. İkinci problemle ilgili örnek de sonra yapılacak. Kitaptaki küp alma örneği iki yöntemi karşılaştırıyor:

```c
#include <stdio.h>

int cubeByValue(int n) {
    return n * n * n;
}

void cubeByReference(int *nPtr) {
    *nPtr = *nPtr * *nPtr * *nPtr;
}

int main() {
    int number = 5;

    number = cubeByValue(number);
    printf("%d\n", number);       /* 125 */

    number = 5;
    cubeByReference(&number);
    printf("%d\n", number);       /* 125 */
    return 0;
}
```

Bu kod ders sonrasında `gcc -Wall` ile derlendi ve iki satırda da 125 yazdırdı. Kodda `number = 5;` yeniden atamasını, ikinci çağrının da 5'ten başladığını göstermek için biz ekledik. Hoca iki

yöntemi ayrı ayrı izledi.

Değerle çağırmanın adımları:

1. `main`'de `number` 5'tir. `cubeByValue`'daki `n`'in içinde ne olduğu henüz bilinmez.
2. `cubeByValue(number)` çağrılınca 5, `n`'e kopyalanır. Bu 5, fonksiyonun yerel kopyasıdır.
3. Fonksiyonda $5 \times 5 \times 5 = 125$ hesaplanır ve `return` ile döndürülür.
4. Fonksiyon işini bitirdiği için içindeki yerel `n` tanımsız hâle gelir.
5. Dönen 125, eşittir operatörüyle `number`'a atanır.

Burada `number` parametre üzerinden değil, `return` üzerinden değişir. Hoca bunu `x = x + 5` ile karşılaştırdı: Önce `x`'in değeri alınır, sonra 5 eklenir, sonra atanır. Burada da önce 5 gönderilir, dönen değer aynı değişkene atanır, çünkü operatörler de değer döndürür.

Pointer ile çağırmanın adımları:

1. `main`'de `number` 5'tir. `nPtr`'nin içeriği henüz tanımsızdır, atama yapılmamıştır.
2. `cubeByReference(&number)` çağrılınca `number`'ın adresi `nPtr`'ye kopyalanır.
3. Sağ taraftaki her `*nPtr` için `nPtr`'nin gösterdiği yere gidilir ve 5 alınır: 5, 5, 5.
4. $5 \times 5 \times 5 = 125$ hesaplanır.
5. Eşittir operatörüyle 125, `nPtr`'nin gösterdiği yere, yani `number`'ın kendisine yazılır.

Hoca "`nPtr`'nin gösterdiği yerdeki değer çarpı `nPtr`'nin gösterdiği yerdeki değer çarpı…" diye okurken bunun tekerleme gibi olduğunu söyledi.

9.3 String Örneği: `convertToUppercase`

Hoca string'lerin pointer'la kullanımını da gösterdi. Normalde string `char kelime[10];` gibi tanımlanıyordu. Karakter pointer'ı da string'lerde kullanılır. String'in sonunu göstermek için son karakterden sonra `'\0'` eklenir. Yani 10 karakterlik bir metin, `'\0'` ile birlikte 11 karakter yer tutar.

> **Ders sonrası not:** Hoca `char kelime[10];` için "10 tane karakter içereceğini söyledim" dedi. `'\0'` da aynı dizide yer aldığı için bu dizide en fazla 9 karakterlik bir metin saklanabilir. Hocanın 11 karakter hesabı, metnin kendisi 10 karakter olduğu durum içindir. Ders sonrasında `sizeof(kelime)` 10 çıktı.

Örnekte string, `convertToUppercase` fonksiyonuna gönderiliyor. Dizi olduğu için zaten call by reference ile gider. Parametre dizi olarak da alınabilirdi, ama burada karakter pointer'ı olarak alınmış. Bunun mümkün olmasının sebebi, dizinin adının zaten adres olmasıdır.

```c
#include <ctype.h>

void convertToUppercase(char *sPtr) {
    while (*sPtr != '\0') {
        if (islower(*sPtr))
            *sPtr = toupper(*sPtr);
        sPtr++;
    }
}
```

`*sPtr`, `sPtr`'nin gösterdiği yerdeki karakterdir. Bu karakter `'\0'`'dan farklı olduğu sürece string'in başından sonuna kadar dolaşılır. `islower` karakterin küçük harf olup olmadığına bakar. Küçükse `toupper` ile büyük harfe çevrilir ve yeni değer aynı yere yazılır. Sonra pointer bir artırılarak sonraki karaktere geçilir. Böylece bütün karakterler büyük harfe çevrilir.

> **Ders sonrası not:** Hoca `islower`'ı "string fonksiyonu" diye tanıttı. Bu fonksiyon ve `toupper` tek bir karakter üzerinde çalışır ve `<ctype.h>` başlığındadır. Yukarıdaki kod bu başlıkla `gcc -Wall` altında uyarısız derlendi ve denenen küçük harfleri büyüğe çevirdi.

Hoca yalnız yazdırma yapan bir fonksiyonda parametreyi `const` tanımlamanın mümkün olduğunu gösterdi, ama `const` konusuna bu derste girmedi. Şu anki odak, adres operatörünün kullanımıdır.

9.4 Bubble Sort'un Pointer Parametreyle Yazılması

Kitapta bubble sort call by reference ile de yazılmış. Diziler zaten bu mekanizmayla gidiyordu, kod da az öncekinin hemen hemen aynısı. Tek fark, dizinin fonksiyonda dizi olarak değil pointer olarak tanımlanmasıdır: `int a[]` yerine `int *a`. Fonksiyon içinde yine normal dizi erişimi gibi `a[j]` kullanılır. Dizi etiketi zaten pointer olduğu için bu mümkündür.

Soru: `int *array` ile `int array[]` aynı mıdır, yıldızın yeri önemli midir? **Cevap:** `int *array` bir pointer tanımıdır. Tür bakımından `array[]` da pointer türündedir, bu yüzden parametre olarak ikisi de sorunsuz çalışır.

Slayttaki `sizeof` örneklerine ve `swap` fonksiyonunun ayrıntılarına hoca girmedi. Bu aşamada önemli olan, operatörlerin fonksiyonlarda, dizilerde ve değişkenlerde kullanımıdır. Hoca, derste bir öğrencinin sorusu üzerine dinamik bellek konusunun bu dersin konusu olmadığını da söyledi.

10. Derste Yazılan Program: Swap, Dizi Yazdırma ve Prototipler

Aradan sonra hoca, buraya kadar söylenenleri yeni bir kaynak dosyada kodlayarak özetledi. Anlatım Dev-C++'ta yapıldı. Program ve aşağıda geçen bölümleri `gcc -Wall` ile derlenip ders sonrasında yeniden çalıştırıldı.

10.1 İki Swap Fonksiyonu

`double x = 5.0, y = 6.0;` tanımlanıyor. Önce `swap_value`, sonra `swap_ref` çağrılıyor, her adımda değerler yazdırılıyor.

```
void swap_value(double x, double y) {
    double t;
    t = x;
    x = y;
    y = t;
}

void swap_ref(double *x, double *y) {
    double t;
    t = *x;
    *x = *y;
    *y = t;
}
```

Soru: `swap_ref`'teki `t` nasıl tanımlanacak? **Cevap:** Pointer tipinde değil. `t` değer saklayacak, yani geçici bir değer tutacak, bu yüzden `double` olur.

`swap_ref`'in okunuşu şöyledir: `x`'in gösterdiği yerdeki değeri `t`'ye kopyala. `x`'in gösterdiği yere `y`'nin gösterdiği yerdeki değeri kopyala. `y`'nin gösterdiği yere de `t`'deki değeri kopyala. Çağrı `swap_ref(&x, &y);` biçimindedir.

Hoca, `swap_value`'daki `x` ve `y`'nin call by value, `swap_ref`'tekilerin ise call by reference olduğunu söyledi. Ama bu gerçek bir call by reference değildir, buna "yalancı call by reference" denebilir. Çünkü aktarılan şey aslında yine bir değerdir: Adres değerleri kopyalanır.

Ekrandaki çıktı (kareden okundu):

```
Before: 5.000000 -- 6.000000
After: 5.000000 -- 6.000000
After-2: 6.000000 -- 5.000000
```

Yani `swap_value` değerleri değiştirmedi, `swap_ref` değiştirdi. Burada ikinci problem de çözülmüş oldu: Hem `x` hem `y` değişti, fonksiyondan bir anlamda iki değer döndürülmüş oldu. `return` ile en fazla biri döndürülebilirdi. Bu yolla iki, üç ya da kaç değer istersek döndürebiliriz.

10.2 Dizi Yazdırma ve Değiştirme

`int A[5] = {1, 2, 3};` tanımlanıyor. İlk üç eleman 1, 2, 3 olur, kalan iki eleman otomatik olarak 0 olur. Sonra iki fonksiyon yazılıyor:

```
void print_array(int *a, int size) {
    int i;
    for (i = 0; i < size; i++)
        printf("%3d", *(a + i));
    printf("\n");
}

void mutate_array(int *a, int size) {
    a[3] = 444;
    size--;
}
```

Burada önemli bir nokta var: `a` için call by reference, `size` için call by value geçerlidir. Her parametrenin gönderilme biçimi kendine özgüdür. Bir parametre referansla gitti diye bütün parametreler referansla gitmiş olmaz. `a`'da yapılan değişiklik yansır, `size`'da yapılan yansımaz.

Soru: `mutate_array` içinde `size--` yapmanın `main`'deki boyuta bir faydası olur mu? **Cevap:** Olmaz. `size` call by value ile gelir, yalnız değeri kopyalanır.

`print_array`, `mutate_array`, `print_array` sırasıyla çağrılınca çıktı `1 2 3 0 0` ve `1 2 3 444 0` olur. Dizinin 3 indisli elemanındaki değişiklik `main`'e yansımıştır. Hoca diziyi değil de dizinin tek bir elemanını gönderseydik değişikliğin yine yansımayacağını hatırlattı, çünkü bu call by value'dur.

10.3 Prototipler ve Derleyici Mesajları

Program ilk derlendiğinde Dev-C++ `conflicting types` uyarıları verdi. Hoca fonksiyonların üstüne prototipleri yazdı. Prototip, derleme aşamasında derleyiciye "ileride bu imzada bir fonksiyon gelecek, haberin olsun" der. Prototipte parametre adları yazılmak zorunda değildir, tipler yazılır: `void print_array(int *, int);`.

```
void swap_value(double x, double y);
void swap_ref(double *x, double *y);
void print_array(int *a, int size);
void mutate_array(int *a, int size);
```

> **Ders sonrası not:** Hoca, işin içine pointer girince prototip yazmanın gerekli olduğunu söyledi. Sorunun sebebi pointer değildir. Kareye göre uyarı, pointer içermeyen `swap_value` için de verilmiş: `conflicting types for 'swap_value'` ve `previous implicit declaration of 'swap_value'`. Fonksiyon tanımından önce çağrılınca derleyici onu örtük olarak bildirir. Sonraki `void` tanım buna uymadığı için uyarı çıkar. Ders sonrasında aynı durum `gcc -Wall -std=c89` ile uyarı, gcc 15'in varsayılan ayarıyla hata verdi.

Hoca ayrıca, fonksiyon `main`'den önce yazılırsa prototipe gerek kalmadığını gösterdi. Bu, bir öğrencinin önceden sorduğu sorulardan biriydi.

Prototip bir bildirimdir (declaration), bir değişken tanımlamak gibi düşünülebilir. Bildirimi yapılan fonksiyon başka bir yerde tanımlanmalıdır. Hoca bunu fonksiyonun adını bozarak gösterdi: `print_k_array` gibi tanımı olmayan bir ada başvurulunca derleme yalnız uyarı verir, fakat sonra `undefined reference` hatası alınır, çünkü böyle bir fonksiyonun tanımı yoktur. Bildirimler ileride başlık (header) dosyalarına alınacak, bu gelecek senenin konusudur.

> **Ders sonrası not:** Hoca bildirimi yapılan şeyin mutlaka tanımlanması gerektiğini söyledi. Bu, fonksiyon çağrılıyorsa geçerlidir. Ders sonrasında `gcc -Wall` ile denendi: Tanımı olmayan bir fonksiyon çağrılınca bağlama aşamasında `undefined reference` hatası alındı. Çağrılmayan bir prototip ise hatasız derlenip bağlandı.

11. Pointer Aritmetiği

11.1 Hazırlık: Dizinin Adını Pointer'a Atamak

Hoca önce normal değişken aritmetiğini hatırlattı: `int a = 5;` için `a++` ya da `a = a + 1`, a'nın içindeki değeri 1 artırır. Sonra `A` dizisi için `int *pa;` tanımladı ve `pa = A;` yazdı. Dizi adının yerine `&A[0]` de yazılabilir.

Soru: `pa = A[0];` yazılabilir mi? **Cevap:** Hayır. `A[0]`'ın içinde 1 vardır, yani bu bir `int`'tir, `pa` ise pointer'dır. Bu atama `pa`'ya 1 numaralı adresi vermek gibi olurdu ve derleyici buna izin vermez. Atamanın iki tarafı aynı türden olmalıdır.

> **Ders sonrası not:** Hoca `pa = &A;` yazımının da olabileceğini söyledi. `&A` aynı adresi verir, ama tipi "5 elemanlı `int` dizisine pointer"dır (`int (*)[5]`). Ders sonrasında bu atama gcc 15'te `incompatible pointer type` derleme hatası verdi. `pa = A;` ya da `pa = &A[0];` yazılmalıdır. Aynı derleyicide `pa = A[0];` de derleme hatası verdi.

11.2 `pa++` Adresi Ne Kadar Artırır?

Hoca bir `printf` ile adresleri ve değerleri yazdırdı. `pa`, `&A[0]` ve `A[0]` için önce sorduğu soru şuydu: Ekranda ne görülecek? Öğrencilerin de söylediği gibi ilk ikisi aynı adres, üçüncüsü ise değer, yani 1 olur. Sonra `pa` artırıldı.

Adres 0x12345 gibi bir değer olsun. Bunu bir artırmak onu 0x12346 yapmaz. **Adres, tanımlı değişkenin boyutu kadar artar.** Yani bir `int` pointer için 1 artırmak, adresi 4 artırmak demektir. Bu sayede `pa` bir sonraki elemanı gösterir, çünkü `int` 4 bayttır ve `A[1]`, `A[0]`'dan 4 bayt sonradır. Karakter olsaydı artış 1'er, 8 baytlık bir tip olsaydı 8'er olurdu. Azaltmalar da buna göre olur. Genel açılım şöyledir:

$$pa + i \implies \text{adres} + i \times \text{sizeof(gösterilen tip)}$$

`pa++` ifadesi `pa = pa + 1` ile aynıdır.

> **Ders sonrası not:** Hoca açılımı `a = a + 1 * sizeof(int *)` olarak yazdı ve `int` pointer'ın boyutunun 4 olduğunu söyledi. Doğrusu, artışın gösterilen tipin boyutu, yani `sizeof(int)` kadar olmasıdır. Hoca az sonra açılımı "`pa` artı 1 çarpı `sizeof(int)`" diye söylüyor. Ders sonrasında bu sistemde `sizeof(int)` 4, `sizeof(int *)` 8 çıktı ve `A + 1` ile `A` arasındaki fark 4 bayt oldu.

Hoca bu sırada kodda `A`'yı artırdığını fark edip düzeltti. Aritmetik `A` üzerinden değil, `pa` üzerinden yapılmalıdır. Çünkü `A` sabittir, dizinin taban adresini tutmak zorundadır, değiştirilemez.

> **Hocanın vurgusu:** Dizinin adı sabittir, her zaman dizinin ilk taban adresini tutar. `A++` yazılamaz. Pointer aritmetiği `pa` gibi ayrı bir pointer değişkeniyle yapılır.

Ders sonrasında `A++` `gcc -Wall` ile denendi: `lvalue required as increment operand` derleme hatası alındı.

11.3 `(*pa)++` Değeri Artırır

Bu noktada `pa`, `A[1]`'i gösteriyor ve `A[1]` 2'dir. Şimdi `pa++` yerine `(*pa)++` yazılıyor.

Soru: Ne olur? **Cevap:** `pa`'nın gösterdiği yerdeki değer 1 artar. 2 olan `A[1]`, 3 olur. Adres değişmez, hâlâ `A[1]`'in adresindeyiz.

Dev-C++ ekranındaki çıktı (kareden okundu), sütunlar sırasıyla iki adres ve iki değerdir:

```
2358800 2358800 1 1
2358804 2358804 2 2
2358804 2358804 3 3
2358808 2358804 3 3
```

- 1. satır: `pa` `A[0]`'ı gösteriyor, değer 1.
- 2. satır: `pa++` sonrası adres $2358800 + 4 = 2358804$ oldu, değer 2.
- 3. satır: `(*pa)++` sonrası adres aynı kaldı, değer 3 oldu.

- 4. satır: Bir sonraki denemeye aittir (Bölüm 11.5).

11.4 Bir Diziyi Yazdırmanın Eşdeğer Yolları

Pointer aritmetiği dizilerde şöyle kullanılır. `print_array`, üç farklı biçimde yazılabilir ve üçü de aynı sonucu verir:

```
#include <stdio.h>

void print_v1(int *a, int size) {
    int i;
    for (i = 0; i < size; i++)
        printf("%3d", a[i]);
    printf("\n");
}

void print_v2(int *a, int size) {
    int i;
    for (i = 0; i < size; i++) {
        printf("%3d", *a);
        a++;              /* yerel a her adimda degisir */
    }
    printf("\n");
}

void print_v3(int *a, int size) {
    int i;
    for (i = 0; i < size; i++)
        printf("%3d", *(a + i));   /* a degismez */
    printf("\n");
}

int main() {
    int A[5] = {1, 2, 3, 444, 0};
    print_v1(A, 5);
    print_v2(A, 5);
    print_v3(A, 5);
    return 0;
}
```

Derste ekran `1 2 3 0 0` ve `1 2 3 444 0` olarak aynı kaldı, ders sonrasında üç fonksiyon da `gcc -Wall` ile derlenip aynı satırı yazdırdı. Farklar şöyledir:

- `*a; a++` biçiminde `a`, fonksiyona ait bir pointer değişkenidir, bu yüzden değiştirilebilir. Ama döngü ilerledikçe fonksiyondaki orijinal adres kaybedilir, çünkü `a` sürekli kendine bir ekleyerek atanır.
- `*(a + i)` biçiminde önce `a`'ya `i` eklenip bir adres üretilir. Sonra `*` ile o adresteki değer getirilir. `a` kendisine atama yapılmadığı için değişmez, adres kaybedilmez. `i` 0 iken `a`'nın kendisi, `i` 1 iken bir sonraki eleman elde edilir.

Adresleri yazdırmak için de `a` (artırılarak), `a + i` ve `&a[i]` eşdeğerdir. Karedeki çıktıda adresler 2358800, 2358804, 2358808, 2358812, 2358816 diye 4'er 4'er ilerliyor ve yanlarında `1 2 3 0 0`, değişiklikten sonra da `1 2 3 444 0` değerleri görülüyor.

> **Hocanın vurgusu:** Pointer aritmetiği, normal değişken aritmetiğinden farklı olarak işin içine boyutu dahil eder. Parantez kullanmak her zaman iyidir, operatör öncelikleri kafa karıştırabilir.
>
> **Ders sonrası not:** Derste adresler `printf`'te `%d` ile yazdırıldı. `gcc -Wall` bu kullanımda `format '%d' expects argument of type 'int'` uyarısı verdi. Adres yazdırmak için `%p` ve `(void *)` dönüşümü kullanıldığında uyarı kalkıyor ve çıktı onaltılık biçimde geliyor. Bu nottaki adres yazdıran kodlarda `%p` tercih edildi. Karedeki ondalık adresler ise derste görülen çıktıdır.

Soru: `print_array` içinde `a`'yı artırmak fonksiyonun dışında etki eder mi? Pointer de kopyalanarak gelmiyor mu? **Cevap:** `main`'deki `A` değişmez. Değer fonksiyona kopyalanır. Hoca önce fonksiyon bitince `a`'nın dizinin sonuna geleceğini söyledi, bu fonksiyon içindeki `a` içindir. Çağrıldığı yerdeki `A` ise bir dizi değişkenidir, ilk elemanın adresini tutar ve hiçbir şekilde değiştirilemez. Fonksiyondaki `a`'nın son değerini dışarıda saklamak mümkündür, ama bu gelecek dönemin konusudur.

11.5 *pa++ Ne Yapar?

Soru: `(*pa)++` yazarken parantez kullanmasak olur muydu? **Cevap:** Bu operatör önceliğiyle ilgilidir. Hoca önceliği o anda hatırlamadığını, emin olmak için parantez kullandığını söyledi ve iki ihtimali anlattı:

- `*` önce çalışırsa: Adresteki değer alınır, o değer bir artırılır.
- `++` önce çalışırsa: Pointer güncellenir, güncellenen yerdeki değer getirilir.

Sonra denedi. `(*pa)++` satırından sonra `*pa++;` yazıp yeniden yazdırdı. Sonuç yukarıdaki 4. satırdır: `2358808 2358804 3 3`. İlk bakışta değer 3 kaldığı için fark görülmedi, bir öğrenci adresin değiştiğini hatırlattı. Gerçekten `pa` 2358804'ten 2358808'e geçmişti. O adreste `A[2]` vardı ve onun değeri de zaten 3'tü. Yani adres artmıştı, değer tesadüfen aynı görünüyordu. Hoca sonucu "`++` önce yapılır, sonra o adresteki değer getirilir" diye özetledi. Bu tür ifadeler yerine önce parantez kullandığını da söyledi, çünkü işin bir sürü ayrıntısı var ve operatör öncelik tablosu uzundur.

> **Ders sonrası not:** Sonuç kısmen doğrudur. Sonek `++`'nın önceliği `*`'tan yüksektir, yani ifade `*(pa++)` olarak okunur. Ama sonek artırma `pa`'nın **eski** değerini kullanır: İfadenin değeri, `pa`'nın artırılmadan önce gösterdiği elemandır. İşlem bitince `pa` bir sonraki elemanı gösterir. Dersteki deneme bu farkı gösteremez, çünkü `*pa++;` satırında okunan değer kullanılmıyor. Ders sonrasında `v = *pa++;` denendi: `pa` `A[1]`'i (3) gösterirken `v` 3 oldu ve `pa` 2 indisli elemana geçti. Ayrıca `gcc -Wall`, `*pa++;` satırı için `value computed is not used` uyarısı verdi. Değeri artırmak için `(*pa)++`, pointer'ı artırmak için `pa++` yazılmalıdır.

Dersin bu bölümündeki programın uyarısız derlenen tam hâli şöyledir. `*pa++;` yerine aynı adres değişikliğini yapan `pa++;` yazıldı:

```c
#include <stdio.h>

void swap_value(double x, double y);
void swap_ref(double *x, double *y);
void print_array(int *a, int size);
void mutate_array(int *a, int size);

int main() {
    double x = 5.0, y = 6.0;
    int A[5] = {1, 2, 3};
    int *pa;

    printf("Before: %lf -- %lf\n", x, y);
    swap_value(x, y);
    printf("After: %lf -- %lf\n", x, y);
    swap_ref(&x, &y);
    printf("After-2: %lf -- %lf\n", x, y);

    print_array(A, 5);
    mutate_array(A, 5);
    print_array(A, 5);

    pa = A;
    printf("\n%p %p %d %d", (void *)pa, (void *)&A[0], A[0], *pa);
    pa++;
    printf("\n%p %p %d %d", (void *)pa, (void *)&A[1], A[1], *pa);
```

```
    (*pa)++;
    printf("\n%p %p %d %d", (void *)pa, (void *)&A[1], A[1], *pa);
    pa++;
    printf("\n%p %p %d %d\n", (void *)pa, (void *)&A[1], A[1], *pa);
    return 0;
}

void swap_value(double x, double y) {
    double t;
    t = x;
    x = y;
    y = t;
}

void swap_ref(double *x, double *y) {
    double t;
    t = *x;
    *x = *y;
    *y = t;
}

void print_array(int *a, int size) {
    int i;
    for (i = 0; i < size; i++)
        printf("%p %3d\n", (void *)(a + i), *(a + i));
    printf("\n");
}

void mutate_array(int *a, int size) {
    a[3] = 444;
    size--;
}
```

Ders sonrasında çıktıdaki değerler karedekilerle aynı çıktı: `Before`, `After`, `After-2` satırları, `1 2 3 0 0`, `1 2 3 444 0`, sonra `1 1`, `2 2`, `3 3`, `3 3` değerleri ve 4'er baytlık adres farkları. Adreslerin kendisi çalıştırmaya göre değişir.

12. Pointer Aritmetiğinin Slayttaki Özeti

Hoca slaytlara dönerek pointer'larla yapılabilen işlemleri özetledi: artırma ve azaltma operatörleri, atama, `+=` ve `-=` biçimindeki atamalar, pointer'dan pointer çıkarma.

- **İlerleme:** Bir dizi kullanıldığında dizinin ilk elemanının adresi pointer'a atanır. Pointer 2 artırılınca doğrudan 2 indisli elemanın adresine gidilir.
- **Karşılaştırma:** Pointer'lar karşılaştırılabilir. Karşılaştırma adresler üzerinden yapılır, hangisinin yüksek numaralı adreste olduğuna göre sonuç alınır.
- **Atama:** Aynı türden pointer'lar birbirine atanır. Tek istisna `void *`'dır. `void` pointer jenerik türdedir: Her türden pointer buna aktarılabilir ve bir tip dönüşümü (typecasting) ile geri alınabilir. Bu gelecek dönem görülecek.

Dizi ile pointer ilişkisinde `bPtr = b;` için `*(bPtr + 3)`, önce 3 eklenip 3 indisli adrese gidilmesi, yani dizinin 4. elemanının yazdırılması demektir. Slayttaki örnekte aynı elemana ofset üzerinden dört farklı yolla erişiliyor:

Gösterim	Anlam
`b[3]`	Dizi gösterimi
`*(b + 3)`	Dizi adı ve ofset. `b` değişmez, yalnız ofset eklenir
`bPtr[3]`	Pointer adıyla köşeli parantez
`*(bPtr + 3)`	Pointer ve ofset

Hepsi aynı sonucu verir, çünkü arka planda köşeli parantez yazımı `*(...)` biçimine dönüştürülür. Ders sonrasında 3 indisli elemanı 444 olan `A` dizisi için bu dört yazım denendi, dördü de 444 verdi.

Hoca kalan slaytları geçti: Karakter kopyalama örnekleri, pointer dizileri ve fonksiyon pointer'ları gelecek dönemin konusudur.

13. Bir Dizinin Sıralı Olup Olmadığını Bulmak

Hoca bu konuları neden anlattığını tekrar etti: Algoritmalar bundan sonra daha çok kod yazılarak işlenecek, akış diyagramı sınırlayıcı kalıyor ve uygulamaya daha yatkın olmak gerekiyor. Sonra ilk algoritma sorusunu sordu: Bir dizinin sıralı olup olmadığı nasıl bulunur? Çözüm öğrencilerle birlikte kuruldu. Burada dizinin küçükten büyüğe sıralı olması aranıyor.

13.1 İlk Çözüm ve Sınır Sorunu

Bir öğrencinin önerdiği çözüm şuydu: `i`'ye 0 ver. `a[i] < a[i + 1]` olduğu sürece `i`'yi artır. Sağlamadığı durumda döngüden çık. Çıkınca `i`'nin eleman sayısına eşit olup olmadığına bak, eşitse "sıralı", değilse "değil" de ve dur.

Sonra `i`'nin sınırına gelindi. Öğrenci sorunu şöyle tarif etti: Dizinin dışına taşmazsa `i`, n'e eşit olamıyor, sınır `i <= n` yazılırsa da `a[i + 1]` dizinin dışına çıkıyor. Doğru sınır, $n-1$ ile ilgilidir.

Hoca sınır kontrolünün `&&`'nin **sol tarafına** yazılması gerektiğini söyledi, çünkü `&&` operatöründe birinci şart yanlışsa diğer şarta hiç bakılmaz. Önce `i < n` şartını yazdı ve son durumda sorun olmadığını düşündü. Ardından gelen bir soru üzerine düzeltti: 0'dan başlanan C kullanımında şart `i < n - 1` olmalıdır, böylece en son $n-2$ ile $n-1$ indisli elemanlara bakılır. Hoca bunu "implementasyon ve C'ye hakim olmak bu yüzden önemli" diye yorumladı.

Son durumda çözüm numaralı adımlarla şöyledir:

1. `i = 0` yap.
2. `i < n - 1` ve `a[i] < a[i + 1]` ise `i`'yi 1 artır ve 2. adıma dön.
3. `i == n - 1` ise "sıralı", değilse "sıralı değil" yaz.
4. Dur.

```
i = 0;
while (i < n - 1 && a[i] < a[i + 1])
    i++;
if (i == n - 1)
    printf("sirali\n");
else
    printf("sirali degil\n");
```

Döngü bittiğinde `i` son indise ulaşmışsa dizi sonuna kadar hiçbir bozukluk görülmemiş demektir. (ara adım eklendi: şart `i < n - 1` olunca döngü sonu kontrolü de `i == n - 1` olur.)

13.2 `i` 1'den Başlatılırsa

Başka bir öğrenci `i`'yi 1'den başlatıp `a[i - 1]` ile `a[i]`'yi karşılaştırmayı önerdi. Hoca bu yaklaşımın doğru ve güzel olduğunu söyledi:

```
i = 1;
while (i < n && a[i - 1] < a[i])
    i++;
if (i == n)
    printf("sirali\n");
else
    printf("sirali degil\n");
```

Soru: `i` en fazla ne olur, son eleman kontrol ediliyor mu? **Cevap:** Karşılaştırmada `i` en fazla $n-1$ olur. O zaman $n-2$ ve $n-1$ indisli elemanlara bakılır, yani son eleman kontrol edilir.

Soru: Bu çözümde son değeri kontrol etmek için şarta `<=` eklemek gerekir mi? **Cevap:** Hayır. Elemanların indisi yine 0'dan başlıyor: Önce `a[0]` ile `a[1]`, en son `a[n-2]` ile `a[n-1]`'e bakılır. Eşittir demeye gerek yoktur.

İki çözüm arasında performans farkı yoktur.

> **Ders sonrası not:** Derste kurulan `a[i] < a[i + 1]` şartı eşit elemanlı sıralı dizileri "sıralı değil" olarak görür. Eşitliğe izin verilecekse şart `a[i] <= a[i + 1]` olmalıdır. Ders sonrasında 1, 2, 2, 3 ile denendi: `<` şartlı iki çözüm "sıralı değil", `<=` şartlı çözüm "sıralı" sonucunu verdi. Aynı deneyde 1, 3, 5, 8 üçünde de "sıralı", 1, 3, 2, 8 üçünde de "sıralı değil" çıktı. Her iki kod `gcc -Wall` ile derlendi.

13.3 Daha Kötü Bir Yöntem

Soru: Bu işin daha kötü bir yöntemi ne olabilir? **Cevap:** Diziyi sıralayıp sonucu göstermek. Bir öğrenci sıralı hâli başka bir diziye koymayı önerdi, hoca dizinin kendi içinde de sıralanabileceğini söyledi. Sıralama için bugüne kadar görülen yöntemlerle n^2 kadar, olsa olsa en iyi ihtimalle $n \log n$ kadar işlem yapılır. Doğrudan kontrol ise n işlemde biter. Aradaki fark ciddidir, bu yüzden doğrudan kontrol daha iyidir.

14. Frekans (Histogram) Hesabı

Son soru şuydu: Bir sayı dizisindeki her bir elemandan kaç tane var? Bu aslında histogramdır. Hoca histogramın olasılık ve istatistik dersinde de karşımıza çıkacağını, iyi bir bilgi çıkarma aracı olduğunu söyledi: Her sayıdan ne kadar var, en fazla ağırlık nerede?

14.1 Birinci Yöntem: İç İçe İki Döngü

Dizimiz 2, 8, 6, 3, 5, 7 olsun, değer aralığı da 0-9 kabul edilsin. Kaç tane 0, kaç tane 1, ..., kaç tane 9 olduğu bulunacak. Sayımlar için yeni bir `B` dizisi kullanılır, başlangıçta bütün değerleri sıfırdır.

Numaralı adımlarla:

1. `B`'nin bütün elemanlarını sıfırla.
2. Dış döngüyle `i`, 0'dan $n-1$'e kadar dizinin elemanlarını gezsin.
3. İç döngüyle `j`, 0'dan 9'a kadar (9 dahil) olası değerleri gezsin.
4. `A[i] == j` ise `B[j]`'yi 1 artır. Değilse hiçbir şey yapma, sonraki `j`'ye geç.
5. İç döngü bitince sonraki `i`'ye geç.
6. Bittiğinde `B`'nin bütün elemanlarını yazdır.

Örneğin `A[i]` 2 iken `j` 2 olduğunda eşitlik sağlanır ve `B[2]` 1 artar.

```c
#include <stdio.h>

int main() {
    int A[6] = {2, 8, 6, 3, 5, 7};
    int B[10] = {0};
    int i, j;

    for (i = 0; i < 6; i++)
        for (j = 0; j <= 9; j++)
            if (A[i] == j)
                B[j]++;

    for (j = 0; j <= 9; j++)
        printf("B[%d] = %d\n", j, B[j]);
    return 0;
}
```

Ders sonrasında derlenip çalıştırıldı: `B[2]`, `B[3]`, `B[5]`, `B[6]`, `B[7]` ve `B[8]` 1, diğerleri 0 çıktı.

Bu iyi bir yöntem mi? Dış döngü n kez döner, her turunda iç döngü de 0'dan 9'a kadar, yani 10 kez döner. Toplam $10n$ işlem vardır. Örnekte $n = 6$ için $6 \times 10 = 60$ karşılaştırma yapılır (ara adım eklendi, python ile sayıldı). Ama değerler rakam olmak zorunda değildir, 1'den 500'e kadar da olabilir. Değer aralığı da yaklaşık n kabul edilirse işlem sayısı $n \times n$, yani n^2'ye yaklaşır. Hoca görüntü işlemeden örnek verdi: Piksel değerleri 0-255 arasında olunca bu yöntem n^2'ye yaklaşan bir maliyet çıkarır. Bundan daha basit bir yöntem vardır.

14.2 İkinci Yöntem: Elemanı İndis Olarak Kullanmak

Fikir şudur: Bir dizinin elemanının değeri, başka bir dizinin indisi olarak kullanılır. Hoca bunun az önceki mod örneğinde de yapıldığını hatırlattı. `A` elemanları, `B` ise frekansları tutar. `if`'e hiç gerek kalmaz:

1. `B`'nin bütün elemanlarını sıfırla. `A`'nın değerleri kullanıcıdan alınmış olsun.
2. `i`, 0'dan $n - 1$'e kadar gitsin.
3. Her adımda `B[A[i]]` değerini 1 artır: `B`'de `A[i]` indisli yere git, oradaki değeri artır.

Hocanın örneği: `A` dizisi 1, 8, 3, 6, 9, 3, 0, 8. `B`'nin boyutu şöyle seçilir: Dizi gezilir, en küçük eleman 0, en büyük eleman 9'dur, o hâlde 0'dan 9'a kadar 10 elemanlı bir `B` dizisi yeter. Gereksiz yer de kaplatılmaz.

Hoca ilk üç adımı gösterdi: `A[0]` 1 olduğu için `B[1]` artar, sonra `B[8]`, sonra `B[3]`. Bütün adımlar şöyledir (ilk üçten sonrası ara adım eklendi, python ve C ile doğrulandı):

i	A[i]	Yapılan işlem	Sonuç
0	1	B[1]++	B[1] = 1
1	8	B[8]++	B[8] = 1
2	3	B[3]++	B[3] = 1
3	6	B[6]++	B[6] = 1
4	9	B[9]++	B[9] = 1
5	3	B[3]++	B[3] = 2
6	0	B[0]++	B[0] = 1
7	8	B[8]++	B[8] = 2

Son durumda `B` dizisi, 0'dan 9'a kadar sırayla `1, 1, 0, 2, 0, 0, 1, 0, 2, 1` olur. Toplam $1 + 1 + 2 + 1 + 2 + 1 = 8$'dir, yani eleman sayısına eşittir.

```c
#include <stdio.h>

int main() {
    int A[8] = {1, 8, 3, 6, 9, 3, 0, 8};
    int B[10] = {0};
    int i;

    for (i = 0; i < 8; i++)
        B[A[i]]++;

    for (i = 0; i < 10; i++)
        printf("B[%d] = %d\n", i, B[i]);
    return 0;
}
```

Burada 0'dan n'e kadar tek döngü vardır, yani n işlem yapılır. Hoca "n^2'den n'e düşmek çok iyi" diye yorumladı.

> **Ek bilgi (derste anlatılmadı):** Bu yöntemde `A`'daki değer doğrudan indis olduğu için değer, `B` dizisinin sınırları içinde olmalıdır. Aralık 0'dan başlamıyorsa, örneğin en küçük değere göre yer ayrılmışsa, indis olarak büyük olasılıkla `A[i] - enKucuk` kullanılır.

14.3 Görüntü (Matris) İçin Frekans

Görüntü bir matristir. Bütün elemanlarını gezmek için mecburen iki döngü gerekir: `i`, 0'dan n'e, `j`, 0'dan m'ye kadar gider. Matris $n \times m$ boyutundadır. `B` yine tek boyutlu frekans dizisidir:

```
for (i = 0; i < n; i++)
    for (j = 0; j < m; j++)
        B[A[i][j]] += 1;
```

`A[i][j]`'deki değer kaçsa, `B`'nin o indisi 1 artırılır. Görüntünün değerleri 0 ile 255 arasında ise `B` dizisi 0'dan 255'e kadar **256** değer içerir. Hoca birinci yöntemi anlatırken "255'lik dizi" demişti, burada doğru sayıyı 256 olarak verdi.

Hoca, tek döngüyle yapılıp yapılamayacağıyla ilgili bir soruya ikinci çözümü göstererek cevap verdi. Slaytlardaki diziler bölümünde de benzer bir örnek olduğunu söyledi: Orada frekanslar hesaplanıp histogram ekrana yıldızlar basılarak gösteriliyor. Bu örneklerin incelenmesini ve bir dizinin ya da matrisin frekansının kodlanmasını istedi.

Dersin Toparlaması

Bu derste önce dizilerin fonksiyonlara nasıl aktarıldığı tekrar edildi: Bütün dizi adıyla gönderildiğinde değişiklik orijinale yansır, tek eleman ise değerle gider. Matrislerde parametreye sütun sayısı yazılmalıdır. Değerle çağırmanın iki sorunu, yani orijinal değerin değiştirilememesi ve en fazla bir değer döndürülebilmesi, pointer'larla çözüldü. C'de bu, adresin kopyalandığı "yalancı" bir call by reference'tır. `&` adresi verir, `*` adresin gösterdiği değeri verir, ikisi birbirini götürür. Pointer aritmetiğinde 1 eklemek, adresi gösterilen tipin boyutu kadar ilerletir, dizinin adı ise sabittir. `a[i]` ile `*(a + i)` aynı şeydir. Dersin sonunda bir dizinin sıralılığını n işlemde kontrol etme ve dizinin elemanını indis olarak kullanıp frekansı n işlemde hesaplama algoritmaları kuruldu.

Hoca, stringler dışında bu dönem için gereken C bilgisinin böylece tamamlandığını, bunun iyi algoritma geliştirip test etmek için yeterli olduğunu söyledi. Yapılar (structure), dosya işlemleri ve dinamik bellek gelecek dönemki C dersinin konusudur. Pointer operatörlerinin, dizi ve değişken aktarımının gözden geçirilmesini istedi, çünkü bunlar hem bu derste hem Sayısal Analiz'de program yazarken işe yarayacak. Sonraki derste bu konulardan yola çıkarak Fatih Hoca'nın ders notlarında da bulunan seçim sistemlerine bakılacak. En basit hâlde adaylara verilen oylar bir dizide tutulur ve en büyüğü bulunarak kazanan belirlenir. Her vatandaşın örneğin en az 3 kişiye oy verdiği durumda ise kimin kime hangi oyu verdiğini saklayacak bir yapı tasarlanacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.